



Frontend Module Introduction

Summer 2026

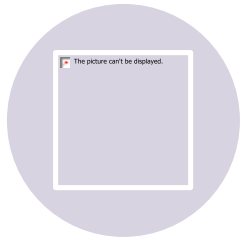
Introduction to frontend



Why is frontend development important?

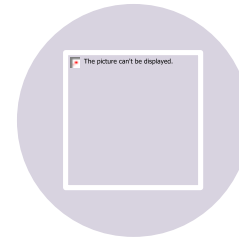
- Leave a lasting first impression.
- Build trust.
- Align with business needs.
- Improve and maintain sales and customer relationships.

Challenges



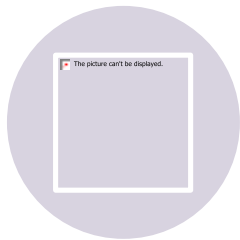
User-friendly

- Performance
- Responsive
- Cross-device
- Cross-browser



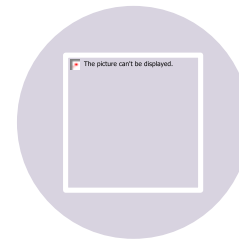
Maintainability

- Scalable
- Easy to handover



Secure

- Compliant to relevant security protocols
- Compliant to common security practices

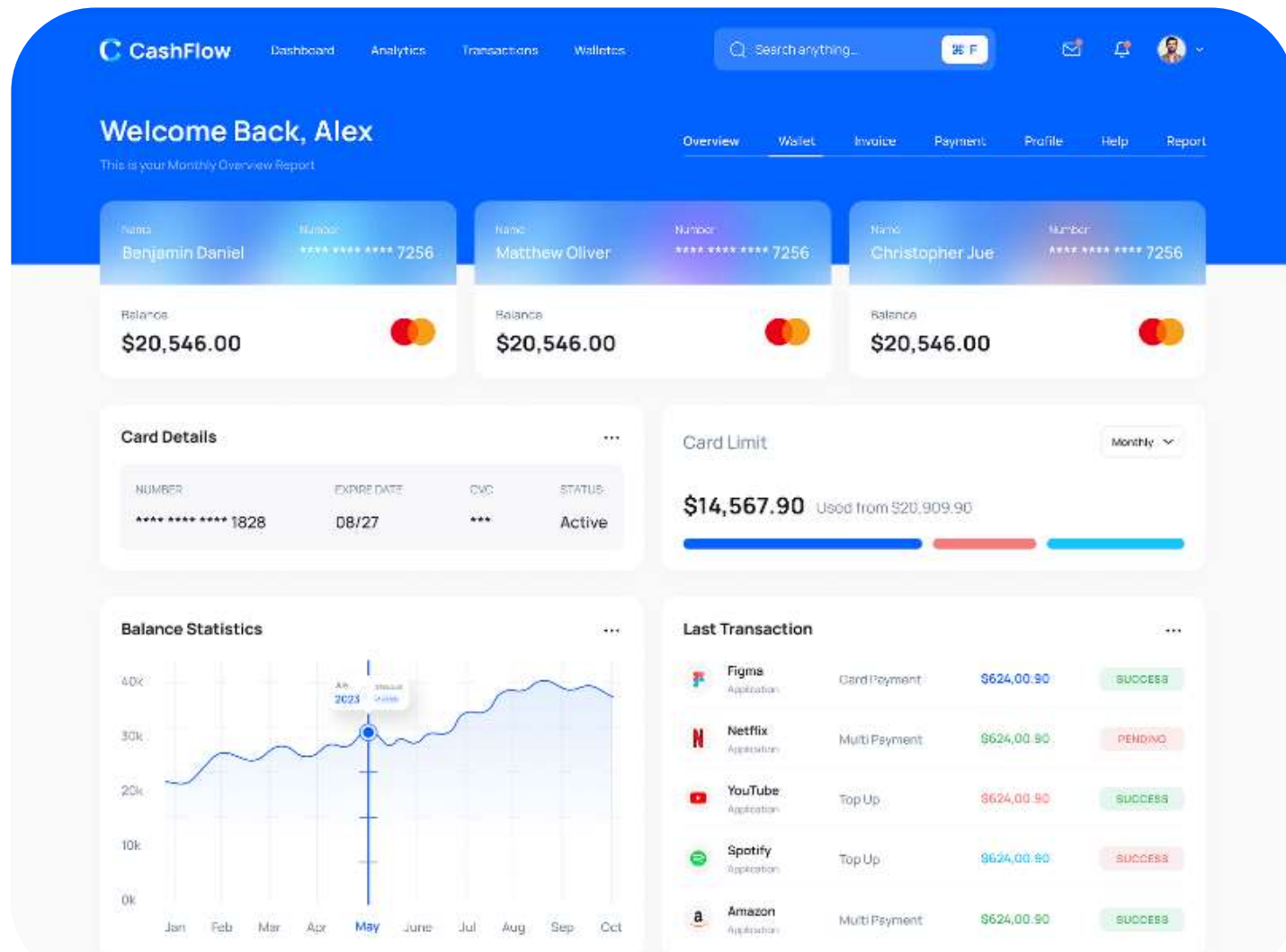


Accessible

- Accommodate for all user groups
- Multilingual

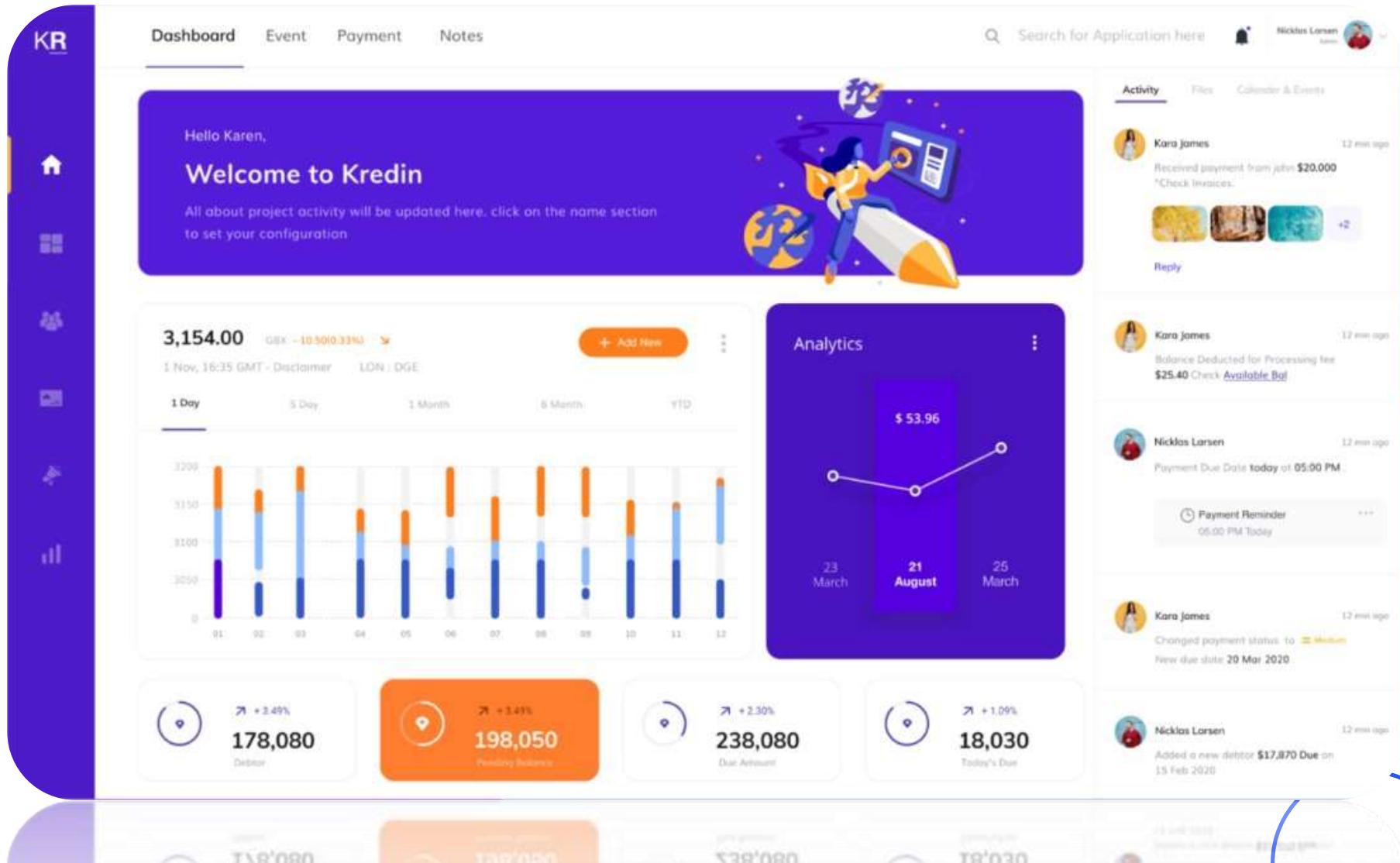
How to look at a page?

- Purpose
- UX/UI
- Layout
- Responsive
- Performance



How to look at a page?

inmind.academy
by inmind.ai



HTML basics

① Syntax to write an HTML element

Opening Tag

Every element has an opening tag with the name of the element at its start.

Attribute and its value (optional)

Attributes are like options of an element. Attributes have value.

```
<name attr="value">
... children
</name>
```

Closing Tag

A closing tag has the name of the element with a forward slash "/" before it.

Children (optional)

Between the opening and closing tags are the children of the element. This can be more elements or just plain text.

```
<name attr="value" />
```

Self-Closing Tag

Some elements that do not have children do not need a closing tag. In this case a forward-slash "/" is added to the element's opening tag to denote a self-closing tag.

Some examples of self-closing elements are:
img, br, hr, meta, input

② Basic markup of every HTML page

```
<!DOCTYPE html>
<html lang="en">
... <head>
... <title>
... page title here
... </title>
... </head>
... <body>
... page content goes here
... </body>
</html>
```

Code Formatting

For good code formatting, remember to indent the children by 2 or 4 spaces.

③ Commonly used HTML elements

<h1> ... <h6>

<p>

<i>

<a>

<div>

lists:

Special formatting

<blockquote> <pre>

multimedia:

 <video>

<audio>

separators:

<hr />

Avoid using videos and audio for better performance

HTML basics

1 `type="text"`
I am a Text Input|

2 `type="password"`
••••|

3 `type="search"`
Search me| X

4 `type="url"`
https://google.com|

5 `type="email"`
hey@example.com|

6 `type="tel"`
+919876543210|

7 `type="number"`
100| :

8 `type="checkbox"`
☐ ☒ ☐ ☐ ☐

9 `type="radio"`
☒ → ☐ ← ☐ ↔

10 `type="range"`

11 `type="button"`
Tweet

12 `type="submit"`
Submit

13 `type="reset"`
Reset

14 `type="file"`
Choose File No File Chosen

15 `type="datetime-local"`
dd/mm/yyyy, --:--:--

16 `type="month"`

17 `type="week"`
Week --, ----

18 `type="time"`
--:--:--

19 `type="date"`
dd/mm/yyyy

20 `type="hidden"`
"input" elements of type "hidden" let web developers include data that cannot be seen or modified by users when a form is submitted.

21 `type="color"`

22 `type="image"`

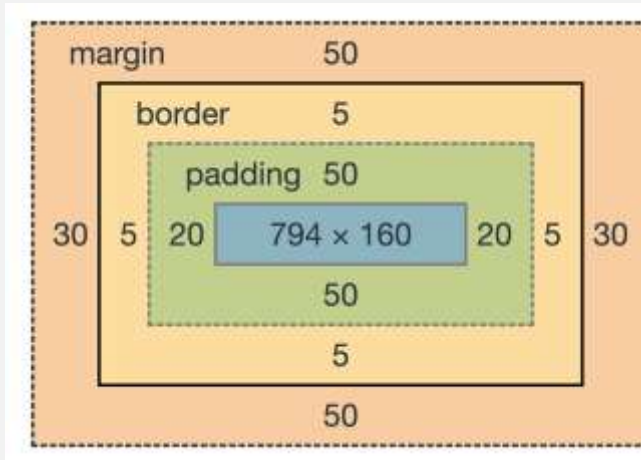
Must-knows: CSS

Patterns used to select the elements you want to style

```
selector {  
  property: value  
}
```

[CSS selectors exercise](#)

Selectors



Box model

Set of rules applied to CSS selectors in order to determine which style is applied to an element

1. Style attribute
2. ID
3. Class, pseudo-class, attribute
4. Element selectors

Specificity

+

Flexbox - Grid

box-sizing VS content-box
?

SCSS VS CSS

```
/* =====  
  VARIABLES  
  ===== */  
$primary-color: #2563eb;  
$danger-color: #dc2626;  
$radius: 8px;  
  
/* =====  
  FUNCTION  
  ===== */  
@function rem($px) {  
  @return ($px / 16) * 1rem;  
}  
  
/* =====  
  CODE REUSE (MIXIN)  
  ===== */  
@mixin button-base($bg) {  
  background-color: $bg;  
  color: #fff;  
  padding: rem(12) rem(20);  
  border-radius: $radius;  
  border: none;  
  cursor: pointer;  
  transition: opacity 0.2s ease;  
  
  &:hover {  
    opacity: 0.85;  
  }  
}
```

```
/* =====  
  NESTING  
  ===== */  
.card {  
  padding: rem(16);  
  border-radius: $radius;  
  background: #f9fafb;  
  
  .card-title {  
    font-size: rem(18);  
    font-weight: 600;  
  }  
  
  .card-actions {  
    margin-top: rem(12);  
  
    .btn-primary {  
      @include button-base($primary-color);  
    }  
  
    .btn-danger {  
      @include button-base($danger-color);  
    }  
  }  
}
```

Typescript

Javascript

- Dynamically typed language.
- Flexibility, but bugs can appear at runtime

Typescript

- Any valid JavaScript code will run as expected in Typescript
- Same syntax as JavaScript + extra features and tools
- Uses static typing, variable types are declared and checked at compile time
- TS needs to be compiled to JS before running in the browser
- Object-Oriented Features

Angular



What is a framework?

- Pre-written app structure to build on top of.
- A collection of files and directories you add your own code and files to.
- Helps build applications quicker by addressing common development problems.



Why Angular?

- Open-source Javascript framework, written in Typescript, maintained by Google.
- Helps build dynamic and interactive Single Page Applications.
- Helps build applications quicker by addressing common development problems and often starts you with boilerplate code & directory structure.



Components

- A component controls a patch of a screen called a view.
- It consists of a Typescript class, an HTML template, and a CSS stylesheet.
- A Component is defined by the class decorated with `@Component()` .



Components

```
import { Component } from '@angular/core';

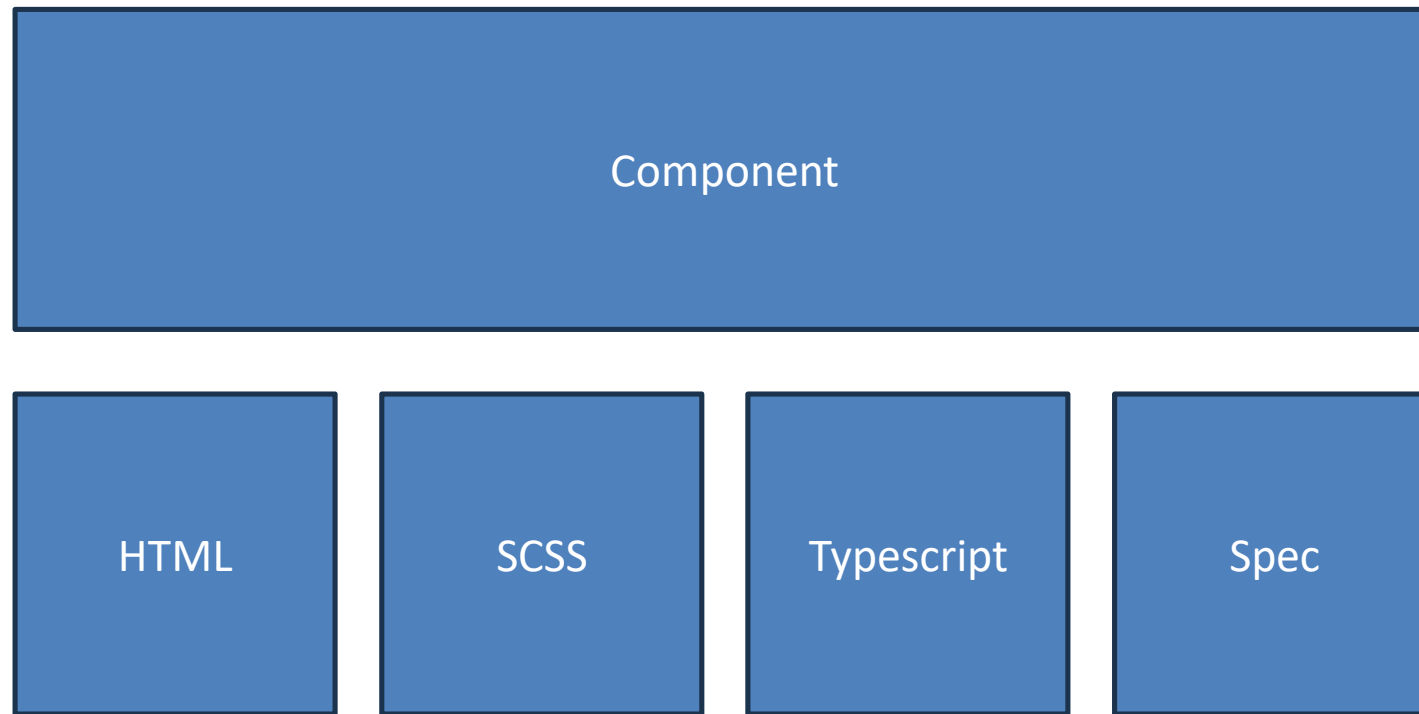
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'angular';
}
```

@Injectable({ providedIn: 'root' })

CONFIGURATION OPTIONS	DETAILS
selector	A CSS selector that tells Angular to create and insert an instance of this component wherever it finds the corresponding tag in template HTML. For example, if an application's HTML contains <code><app-root></app-root></code> , then Angular inserts an instance of the AppComponent view between those tags.
templateUrl	The module-relative address of this component's HTML template. Alternatively, you can provide the HTML template inline, as the value of the template property. This template defines the component's host view.
providers	An array of providers for services that the component requires. In the example, this tells Angular how to provide the HeroService instance that the component's constructor uses to get the list of heroes to display.

Angular building blocks:

Components



Modules

- The app can be split into independent pieces.
- You can add, remove, or change parts without breaking the rest of the system.
- Module in Angular refers to a container that groups similar components, directives, pipes, services... which do a certain task.
- An NgModule is defined by the class decorated with `@NgModule()`.

Modules

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

@NgModule({
  imports:    [ BrowserModule ],
  providers:  [ Logger ],
  declarations: [ AppComponent ],
  exports:    [ AppComponent ],
  bootstrap:  [ AppComponent ]
})
export class AppModule { }
```

```
export class AppModule { }
```

PROPERTIES	DETAILS
declarations	The components , directives, and pipes that belong to this NgModule.
exports	The subset of declarations that should be visible and usable in the component templates of other NgModules.
imports	Other modules whose exported classes are needed by component templates declared in this NgModule.
providers	Creators of services that this NgModule contributes to the global collection of services; they become accessible in all parts of the application. (You can also specify providers at the component level.)
bootstrap	The main application view, called the root component, which hosts all other application views. Only the root NgModule should set the bootstrap property.

From Modules to Standalone Components

```
@Component({
  standalone: true,
  selector: 'app-root',
  imports: [
    RouterModule,
    HomeComponent,
    AboutComponent,
  ],
  templateUrl: '...'
})
export class AppComponent {
  [...]
}
```



```
@NgModule({
  imports: [
    RouterModule,
    HomeComponentModule,
    AboutComponentModule,
  ],
  declares: [
    AppComponent
  ]
})
export
```

```
@Component({
  selector: 'app-root',
  templateUrl: '...'
})
export class AppComponent {
  [...]
}
```

Angular Flow

- **angular.json**: file that is first referred to by the builder to look for all the paths and configurations and to check which is the main file.
- **main.ts**: this file acts as the entry point of the application. **main.ts** helps in creating the browser environment for the application to run, the file then calls the `bootstrapModule(AppModule)` which tells the builder to bootstrap the app.
- **app.module.ts**: from the main we are bootstrapping the AppModule, this module contains the declarations of all the components we are creating so that angular is aware of them.
- **app.component.ts**: the module asks to bootstrap the AppComponent. This is the file that interacts with the html of the webpage and serves it with the data.

Angular Flow

By this time, the compiler has all the details about the components of the app and now they are ready to be used.

- **index.html:** angular is well aware of the modules, components, styles, scripts which are required to display the page so the index.html file is called. The compiler dynamically adds the JS files at the end of this file. Since all the components are now known, the html file calls the root component that is **app-root** which targets **app.component.html**.

Angular other files

- **.editorconfig:** configuration of the code editor. Thanks to it, developers working in different editors can work on a uniform-looking code style.
- **.gitignore:** specifies which files git should ignore
- **README.md:** description of project in the Markdown language.
- **tsconfig.json:** configures TypeScript behavior for the project.
- **package.json:** contains project metadata and dependencies: project name, version, scripts (start, build, test), and libraries used.
- **package-lock.json:** info about all packages installed in node_modules.

Suggested Architecture

- Locate: Make locating code easy and intuitive .
- Identify: The file name must allow you to know what it contains.
- Flat: Keep the folder structure as flat as possible.
- Try to be DRY: Don't Repeat Yourself. This is important but not crucial sometimes it would make your work easier to repeat some things.
- Avoid dependencies between features
- Use lazy loading

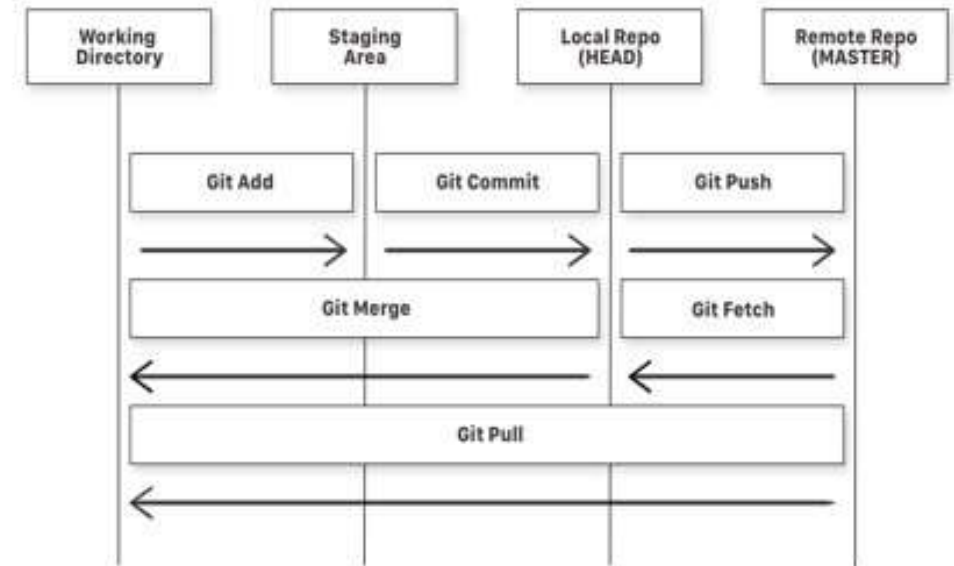
```
/app
├── core/
│   ├── auth/           // Singleton auth service, guards, interceptors
│   ├── layout/         // Main layout, header, sidebar, etc.
│   └── core.module.ts
├── pages/
│   ├── home/
│   │   ├── home.component.ts
│   │   ├── home.module.ts
│   │   └── home-routing.module.ts
│   ├── product-list/
│   │   ├── product-list.component.ts
│   │   ├── product-list.module.ts
│   │   └── components/
│   │       └── product-card/
│   ├── product-detail/
│   │   ├── product-detail.component.ts
│   │   └── product-detail.module.ts
│   ├── login/
│   │   ├── login.component.ts
│   │   └── login.module.ts
├── shared/
│   ├── components/     // Generic components (e.g., button, input)
│   ├── directives/
│   ├── pipes/
│   ├── services/       // Reusable non-singleton services
│   ├── interfaces/
│   └── utils/
├── app-routing.module.ts
└── app.component.ts
```

Git



What is Git?

- Version control system used for tracking code changes and coding collaboration.
- Every project has its own repository: a collection of source code.
- We will be using **GitHub** as the code hosting platform for version control and collaboration. Allowing us, through its tools to integrate with Git



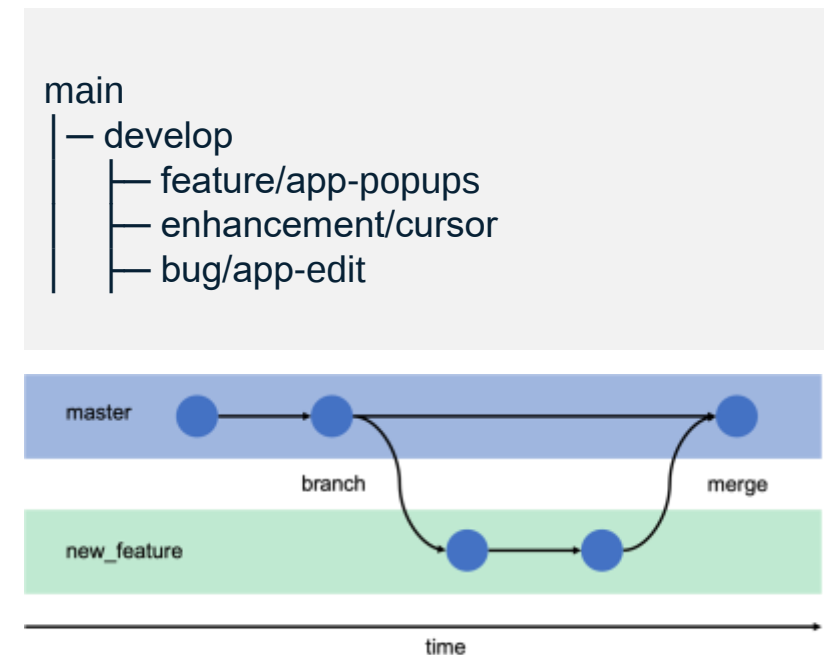
Branching Strategy

- **main** contains the latest stable version of the application - *only pushed to when all changes have been tested, approved and done*
- **develop** contains the latest changes of the application - *pushed to after the current changes have been locally tested and ready to be seen online*

Whenever a new feature, enhancement or bug fix needs to be introduced:

Branch out from the latest version of develop

- **feature/{{feature-name}}**
- **enhancement/{{enhancement-name}}**
- **bug/{{bug-name}}**



Pull Request

- Local changes done, tested, committed and pushed => Next step: merge code with that on develop to view the changes online .
- In order to merge: a pull request should be created, reviewed and accepted. On PR creation, Git will let you know whether your branch is ready to merge or whether there are merge conflicts .
- A merge conflict is an event that occurs when Git is unable to automatically resolve differences in code between two commits. In such a case, you would have to manually resolve those conflicts instead (by going back to your branch, running git pull develop and following the IDE's steps) .
- Code reviews include: naming conventions, styling conventions, optimizations, code is clear and readable, logic errors.

Devtools



What are Devtools?

Tools built right into all major browsers that provide:

- **Elements:** Representation of the page structure in HTML.
- **Console:** A way to view Javascript logs and try out commands on the page.
- **Network:** A way to monitor incoming and outgoing HTTP requests of a web page.
- **Sources:** Lets you **view, inspect, and debug the code and scripts** that the browser is running.
- **Lighthouse & Performance Insights:** Tools to monitor the performance of your website in order to increase loading speed.
- **Toggle device toolbar:** The ability to switch to a mobile view of your website to test the responsive design.

Elements

- See and manipulate elements of the page.
- Hover over any element to have it highlighted on the page and view all its CSS properties.
- Experiment with CSS.
- Force a state on an element (hover, focus, active, visited...) and view its effects.
- View the box model.

 **All changes made are only visible to you. To make changes permanent: mark, copy and paste the modified CSS rules .**



Console

- Find all messages sent to the log.
- View thrown errors.
- Experiment with Javascript inside the console.

Network

- See if and how resources are being uploaded or downloaded.
- Check properties of individual resource files.
- View a waterfall diagram.
- Simulate different network connection speeds.

Sources

- Browse site files.
- Read and edit code.
- Use built-in debugging tools to insert breakpoints which allow you to pause the code execution during certain events to see if an error occurs.
- Use built-in debugging tools to step through the code one line at a time

Performance & Optimization

- Coverage tab: find unused code that can be removed to make the site faster.
- Lighthouse tab: analyze website speed, and run all kinds of audits on your sight.
- Performance Insights: get information on what is and isn't optimized on your page and how to improve it.

Toggle device toolbar

- See what your site looks like on smaller screens and input dimensions numerically in the top bar.
- Choose devices to get their exact screen dimensions.
- Use the icon on the right to switch the screen orientation.
- Adjust the zoom level using the percentage.
- Simulate different connection speeds using the drop-down that says “No throttling”.

Next steps

Moving Forward

- Follow code best practices.
- Put yourself in the shoes of the user .
- Keep in mind that you're part of a team.
- It's not rocket science.

Homework

- Follow the tutorial:
<https://v20.angular.dev/>
- Finalize unfinished in-session work

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld.
757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA

